

B ve B+ Ağaçları

Modern Veritabanları ve Depolama Sistemlerinde Kullanım

Uygulamalar ve Günümüzdeki Önemi

B ve B+ Ağaçları Nedir?

B-Tree

- Kendiliğinden dengeli ağaç yapısı
- Her düğümde birden fazla anahtar
- Veri her seviyede depolanabilir
- Arama, ekleme, silme: $O(\log n)$
- Disk I/O için optimize edilmiş

B+ Tree

- Tüm veri yaprak düğümlerde bulunur
- İç düğümler sadece indeksleme
- Yapraklar birbirine bağlıdır
- Aralık sorguları için mükemmel
- Sıralı erişim için optimize

Gerçek Hayat Örneği: Hastane Hasta Dosyaları

Bir hastanenin 50 yıl içinde accumüle etmiş olduğu 2 milyon hasta dosyasını hayal edin. Bu dosyaları fiziksel dosya dolaplarında organize edip, hastalardan birinin kaydını bulmanız gerekiyor.

B+ Tree Olmadan

2 Milyon Hasta Dosyası

Tüm dosyaları sırayla arama:

$2.000.000 \text{ dosya} \times 1 \text{ sn} =$

23 GÜN

X Zaman kaybı

X maliyet artışı

X Veri çok ise dizinde ayrı bir depo gerekir

B+ Tree ile

3 Seviye İndeksleme

$\log_{500}(2.000.000) \approx 2\text{-}3 \text{ seviye}$

Maksimum 3 dolap açılması

3-5 SANİYE

✓ Çok hızlı

✓ Verimli

✓ Yapraklar bağlıdır

Büyük Veritabanlarında Kullanımı

MySQL

InnoDB Motoru

- Her indeks için B+ tree
- Primary key clustered index
- Hızlı aralık sorguları

PostgreSQL

BTREE İndeksi

- Varsayılan indeks türü
- Verimli sıralama
- Karşılaştırma işlemleri

MongoDB

B-tree İndeksleme

- Her dokuman indekslenir
- Compound indexler
- Sharding desteği

Dosya Sistemlerinde Kullanımı

Windows NTFS

Master File Table (MFT) B+ tree yapısı kullanır

Linux ext4

Extent tabanlı ayırma B-tree indeksleme ile

macOS HFS+

Katalog dosyası B-tree yapısı kullanır

Linux Btrfs

Adından da anlaşılacağı gibi B-tree tabanlı dosya sistemi

Arama Motorları ve Büyük Veri Sistemleri

Google Arama

- Google'ın web indexi B-tree benzeri distributed tree yapılara dayanır
- MapReduce ve BigTable gibi sistemlerde B-tree varyasyonları kullanılır
- Trilyonlarca web sayfası işlenirken verimlilik B-tree sayesinde mümkün

Elasticsearch

- Inverted indexes: Kelime → dokümanlara geri bağlantı
- Bu indexler B+ tree benzeri ağaç yapılarında saklanır
- Milyarlarca dokümanda milisaniye içinde arama yapılabilir

Neden Modern Sistemlerde Tercih Ediliyor?

Disk I/O Verimliliği

Disk bloğu boyutuna optimized. Disk erişim sayısını minimize eder.

Logaritmik Karmaşıklık

$O(\log n)$ arama milyarlar için. 1 milyar kayıt = ~30 disk erişimi.

Kendiliğinden Dengeli

Ekleme/silme sırasında otomatik balance. Yeniden indeksleme yok.

Aralık Sorgusu Desteği

B+ tree yaprakları bağlıdır. Sıralı erişim ve aralık sorguları mükemmel.

Muazzam Ölçeklenebilirlik

Terabyte'tan petabyte'a. Derinlik sadece $\log(n)$ kadar artar.

Büyük Veri Çağında Neden Gerekli?

Çözdükleri Sorun

**Günde 2.5 Exabyte
new data oluşturuluyor**

B-tree olmadan:

Yönetmek imkansız

B-tree ile:

TÜM veriye anında erişim

Kullanıcı Beklentileri

Gerçek Zamanlı Sonuç

Milisaniye içinde cevap

Dağıtık Sistemler

Multi-node DB'lerde hızlı indeksleme

24/7 Kullanılabilirlik

B-tree updates sırasında performans tutar

Gelecekte de Kullanılacak mı?

✓ EVET - Kesinlikle Kullanılacaktır

1. Matematiksel Optimality: B-tree disk I/O için kanıtlanmış optimal yapı. Daha iyi çözüm yok.
2. Quantum Bilgisayarlar Yardım Etmez: Quantum bile disk I/O sorununu çözemez.
3. Muazzam Veri Büyümesi: Veri her 2-3 yılda ikiyle katlanır. B-tree $O(\log n)$ ile mükemmel ölçeklenir.
4. Yeni Teknoloji Adaptasyonu: SSD, NVMe, Memory-mapped I/O - hepsi B-tree indekslemeden faydalanır.
5. Endüstri Standardı: MySQL, PostgreSQL, MongoDB - milyonlarca şirket tarafından kullanılır.
6. LSM Tree B-tree Üzerine İnşa Edilir: Yeni indeksleme yöntemleri B-tree varyasyonlarıdır.

Sonuç

B ve B+ Ağaçları modern teknolojinin **sessiz temel**idir.

Google **trilyonlarca sayfayı indekslemesinden**

Facebook **milyarlarca kullanıcıyı yönetmesine,**

Spotify **trilyonlarca şarkı erişimi sağlamasına kadar:**

Hepsinin altında B-tree teknolojisi vardır.

50+ yıl sonra hala vazgeçilmezdir.